

Shakespeare Sonnet Generation Task: Sentiment Trajectory Guidance 실험 설계

현재 과제의 기본 sonnet generation task는 GPT-2가 Shakespeare sonnet의 첫 3줄을 입력으로 받고, 나머지 11줄을 생성하는 구조이다. 기존 모델은 단순히 next-token prediction loss만 사용하기 때문에, 생성된 시가 Shakespeare sonnet 특유의 감정 흐름이나 volta 구조를 잘 따르는지는 직접적으로 학습하지 않는다.

이번 추가 실험에서는 **Shakespeare's Sonnets Volta & Sentiment Dataset**을 활용해, 각 sonnet의 line-by-line sentiment trajectory를 생성 과정에 반영할 수 있는지 확인하고자 한다. 해당 데이터셋은 Shakespeare의 154개 sonnet 각 line에 대해 positive, neutral, negative에 해당하는 sentiment score를 부여한 데이터셋이며, 논문에서는 sonnet을 14개의 sentiment score sequence로 보고 volta를 분석한다.

1. Research Question

Can sentiment trajectory guidance improve GPT-2 sonnet generation?

즉, GPT-2가 첫 3줄만 보고 나머지 11줄을 생성할 때, sonnet의 감정 곡선 정보를 함께 제공하면 더 구조적이고 감정 흐름이 자연스러운 sonnet을 생성할 수 있는지 확인한다.

2. 데이터셋 및 전처리

이번 실험에서는 두 개의 서로 다른 데이터셋을 사용하게 된다.

1. 현재 과제에서 사용하는 Shakespeare Sonnet Dataset

2. GPT-2 sonnet generation 학습 및 평가에 사용

3. train / validation / test split이 이미 존재

4. Shakespeare's Sonnets Volta & Sentiment Dataset

5. Shakespeare의 154개 sonnet에 대한 line-level sentiment trajectory 제공

6. 각 sonnet은 14개의 sentiment score sequence를 가짐

문제는 두 데이터셋이 독립적으로 구성되어 있기 때문에, 현재 generation task에서 사용하는 sonnet과 sentiment dataset의 sonnet을 매칭하는 과정이 필요하다는 점이다.

Step 1. Sonnet Matching

먼저 두 데이터셋의 sonnet을 sonnet 번호 또는 원문 텍스트 기준으로 매칭한다.

```
Sonnet #1
→ poem text
→ sentiment trajectory

Sonnet #2
```

```
→ poem text
→ sentiment trajectory
```

최종적으로 다음과 같은 형태의 데이터를 구축한다.

```
{
  sonnet_id,
  poem_text,
  sentiment_trajectory
}
```

Step 2. Test Leakage 방지

가장 중요한 점은 sentiment 정보를 사용할 때 test set 정보가 학습 과정에 유출되지 않도록 하는 것이다.

현재 generation task에서 사용하는 test sonnet이 있다고 가정하면,

```
Generation Dataset
├─ Train
├─ Validation
└─ Test
```

sentiment dataset에서도 동일한 sonnet을 찾아 분리한다.

```
Sentiment Dataset
├─ Train Sentiment
├─ Validation Sentiment
└─ Test Sentiment
```

즉,

```
Generation Test Sonnet
=
Sentiment Test Sonnet
```

이 되도록 맞춘다.

Step 3. Sentiment 학습 데이터 구축

Predicted Sentiment Guidance를 수행할 경우에는 sentiment planner를 별도로 학습해야 한다.

이때 planner는 다음 데이터만 사용한다.

```
Generation Train Sonnets
+
Generation Validation Sonnets
```

에 대응되는 sentiment trajectory만 사용한다.

반대로,

```
Generation Test Sonnets
```

에 해당하는 sentiment trajectory는 planner 학습에 절대 사용하지 않는다.

즉,

```
Planner Train
=
Train Sentiment + Validation Sentiment

Planner Test
=
Test Sentiment
```

구조를 유지한다.

이를 통해 planner가 test sonnet의 감정 곡선을 미리 학습하는 데이터 누수(data leakage)를 방지할 수 있다.

Step 4. Average Sentiment 계산

Average-guided 실험에서는 평균 sentiment trajectory를 계산해야 한다.

이때 평균 trajectory는 반드시 train 데이터에서만 계산한다.

```
[
\bar{S} = \frac{1}{N} \sum_{i=1}^N S_i
]
```

여기서 (S_i)는 train set에 포함된 sonnet의 sentiment trajectory이다.

Validation 및 Test sonnet의 sentiment는 평균 계산에 포함하지 않는다.

Step 5. Oracle Sentiment 구성

Oracle-guided 실험에서는 test sonnet의 실제 sentiment trajectory를 입력으로 제공한다.

```
Gold sentiment trajectory
+
First 3 lines
→ GPT-2
```

이 방식은 실제 사용 가능한 설정은 아니지만, sentiment trajectory 자체가 generation에 얼마나 도움이 되는지를 확인하기 위한 upper-bound 실험으로 사용한다.

3. 기본 아이디어

기존 입력은 다음과 같다.

```
<POEM>
first line
second line
third line
```

Sentiment-guided 방식에서는 입력 앞에 감정 곡선을 추가한다.

```
<SENT> 1 1 0 -1 -1 0 1 ... </SENT>
<POEM>
first line
second line
third line
```

여기서 sentiment trajectory는 한 sonnet의 14개 line에 대응하는 감정 점수 sequence이다.

```
[
S = (s_1, s_2, \dots, s_{14})
]
```

모델은 다음 조건부 확률을 학습한다.

```
[
P(\text{remaining 11 lines} \mid \text{first 3 lines}, \text{sentiment trajectory})
]
```

4. 실험 방법 후보

Method A. Oracle Sentiment Guidance

Test sonnet의 실제 정답 14줄 sentiment trajectory를 입력으로 제공한다.

이 방식은 실제 서비스에서는 사용할 수 없지만, 실험적으로 중요하다. 왜냐하면 “정답 감정 곡선을 알면 GPT-2 생성 품질이 좋아지는가?”를 확인할 수 있기 때문이다.

비교 결과 Oracle-guided 모델이 baseline보다 좋아진다면, sentiment trajectory 자체가 generation에 유용한 signal이라는 근거가 된다.

Method B. Average Sentiment Guidance

Train set 전체 sonnet의 평균 sentiment trajectory를 계산한다.

$$\begin{aligned} &[\\ \bar{S} &= \frac{1}{N} \sum_{i=1}^N S_i \\ &] \end{aligned}$$

그 후 test prompt에는 모든 sonnet에 같은 평균 감정 곡선을 제공한다.

이 방식은 test 정답을 몰라도 사용할 수 있다. 즉, Shakespeare sonnet의 평균적인 감정 구조를 generation prior로 주는 방식이다.

Method C. Predicted Sentiment Guidance

별도의 sentiment planner를 학습한다.

```
first 3 lines
→ sentiment planner
→ predicted 14-line sentiment trajectory
```

그 다음 generator는 다음 입력을 받는다.

```
predicted sentiment trajectory + first 3 lines
→ remaining 11 lines
```

전체 구조는 다음과 같다.

```
[Sentiment Planner]
first 3 lines → predicted sentiment trajectory

[Sonnet Generator]
first 3 lines + predicted trajectory → remaining 11 lines
```

이 방식이 가장 현실적인 최종 구조이다. 다만 구현 난이도가 높기 때문에, 먼저 Oracle-guided와 Average-guided 실험을 통해 sentiment trajectory가 유용한지 확인한 뒤 진행하는 것이 좋다.

5. 비교 모델

실험에서는 최소한 다음 모델들을 비교한다.

Model	Input	Purpose
Baseline GPT-2	first 3 lines	기존 방식
Average-guided GPT-2	average sentiment trajectory + first 3 lines	평균 감정 구조의 효과 확인
Oracle-guided GPT-2	gold sentiment trajectory + first 3 lines	감정 곡선 정보의 최대 효과 확인
Predicted-guided GPT-2	predicted sentiment trajectory + first 3 lines	실제 사용 가능한 pipeline 확인

6. 학습 방식

가장 단순한 방식은 GPT-2의 loss를 바꾸지 않고 입력 형식만 바꾸는 것이다.

기존 학습 sample:

```
From fairest creatures we desire increase,  
That thereby beauty's rose might never die,  
...
```

변경된 학습 sample:

```
<SENT> 1 1 0 -1 -1 0 1 ... </SENT>  
<POEM>  
From fairest creatures we desire increase,  
That thereby beauty's rose might never die,  
...
```

Loss는 기존과 동일하게 cross-entropy loss를 사용한다.

```
[  
L = L_{CE}  
]
```

즉, 모델이 sentiment prefix를 조건 정보로 활용하도록 학습시키는 방식이다.

7. 평가 지표

단순 chrF만으로는 시의 감정 구조를 평가하기 어렵기 때문에 다음 지표를 함께 사용한다.

1. chrF

생성 결과와 reference sonnet의 표면적 유사도를 측정한다.

2. Sentiment MSE

생성된 sonnet의 line-by-line sentiment trajectory와 target trajectory 사이의 평균제곱오차를 측정한다.

```
[  
MSE(S_{gen}, S_{target})  
]
```

낮을수록 target 감정 곡선을 잘 따른 것이다.

3. Sentiment Correlation

생성된 감정 곡선과 target 감정 곡선의 흐름이 비슷한지 측정한다.

```
[  
Corr(S_{gen}, S_{target})  
]
```

높을수록 좋다.

4. Volta Distance

Volta를 sentiment 변화가 가장 큰 지점으로 정의한다.

```
[  
v = \arg\max_i |s_{i+1} - s_i|  
]
```

그리고 reference volta와 generated volta의 거리 차이를 측정한다.

```
[  
|v_{ref} - v_{gen}|  
]
```

낮을수록 sonnet의 감정 전환 위치를 잘 따른 것이다.

5. Distinct-2

생성 문장에서 unique bigram 비율을 측정하여 반복이 줄었는지 확인한다.

8. 실험 순서

실험은 다음 순서로 진행하는 것이 가장 합리적이다.

Phase 1. 데이터 매칭 및 전처리

1. Generation dataset과 sentiment dataset 매칭
2. Sonnet ID 또는 원문 기준 정렬
3. Train / Validation / Test split 동기화
4. Test leakage 여부 확인

Phase 2. Baseline 구축

```
First 3 lines
→ GPT-2
→ Remaining 11 lines
```

Baseline 성능 측정

Phase 3. Average-guided 실험

```
Average sentiment trajectory
+
First 3 lines
→ GPT-2
```

평균 감정 구조가 generation에 도움이 되는지 확인

Phase 4. Oracle-guided 실험

```
Gold sentiment trajectory
+
First 3 lines
→ GPT-2
```

감정 곡선 정보의 최대 효과 확인

Phase 5. Predicted-guided 실험

```
First 3 lines
→ Sentiment Planner
→ Predicted Trajectory

Predicted Trajectory
+
```


First 3 lines
→ GPT-2

실제 사용 가능한 sentiment planning pipeline 구축

9. 예상 가설

이 실험의 가설은 다음과 같다.

1. Oracle-guided GPT-2는 baseline보다 sentiment MSE가 낮고, sentiment correlation이 높을 것이다.
 2. Average-guided GPT-2가 baseline보다 좋아진다면, Shakespeare sonnet의 평균 감정 구조가 generation prior로 작동할 수 있다.
 3. Predicted-guided GPT-2가 baseline보다 좋아진다면, 첫 3줄만으로 감정 곡선을 예측하고 이를 generation에 활용하는 pipeline이 가능하다는 것을 보일 수 있다.
 4. chrF는 크게 향상되지 않을 수도 있지만, sentiment trajectory metric과 volta distance에서는 향상이 나타날 수 있다.
-

10. 실험의 의의

이 실험은 GPT-2 sonnet generation을 단순 next-token prediction 문제가 아니라, **시의 감정 구조를 조건으로 하는 generation task**로 확장한다는 점에서 의미가 있다.

특히 기존 평가 방식은 생성된 시가 reference와 얼마나 표면적으로 비슷한지만 보는 경향이 있다. 하지만 시에서는 단어 overlap보다 감정의 흐름, 전환, 구조가 중요할 수 있다. 따라서 sentiment trajectory를 generation signal과 evaluation metric으로 활용하면, sonnet generation의 품질을 더 문학적인 관점에서 분석할 수 있다.